

Strumenti

Autoload e namespace

Come organizzare file, classi e nomi in progetti PHP più grandi.

Il problema dei file

In un progetto piccolo puoi usare `require` per includere file a mano. In un progetto più grande diventa scomodo.

L'autoload carica le classi quando servono, senza scrivere `require` per ogni file.

Namespace

Un namespace è uno spazio dei nomi. Serve a evitare conflitti tra classi con lo stesso nome.

```
<?php
namespace App\Model;

class Prodotto {
}
```

Questa classe si chiama davvero `App\Model\Prodotto`.

use

Per usare una classe con namespace puoi importarla con `use`.

```
<?php
use App\Model\Prodotto;

$prodotto = new Prodotto();
```

Senza `use` , dovresti scrivere il nome completo.

Struttura delle cartelle

Una struttura comune e:

```
progetto/
  src/
    Model/
      Prodotto.php
  vendor/
  composer.json
```

Il file `Prodotto.php` contiene la classe `App\Model\Prodotto` .

Autoload con Composer

Nel `composer.json` puoi configurare l'autoload:

```
{
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  }
}
```

Poi esegui:

```
composer dump-autoload
```

Nel codice principale carichi l'autoload:

```
<?php  
require __DIR__ . '/vendor/autoload.php';
```

Nomi coerenti

Usa nomi di classi chiari e una cartella coerente con il namespace. Questo rende il progetto più facile da navigare.

Un file, una classe principale

Una convenzione semplice è mettere una classe principale per file.

```
<?php  
namespace App\Model;  
  
class Prodotto {  
}
```

Questo file potrebbe chiamarsi `src/Model/Prodotto.php`. Il percorso rispecchia il namespace.

Perche serve dump-autoload

Quando cambi la configurazione dell'autoload in `composer.json`, Composer deve rigenerare le informazioni interne.

```
composer dump-autoload
```

Se aggiungi una nuova classe seguendo la struttura già configurata, spesso non serve fare altro. Se invece cambi la mappa dei namespace, esegui questo comando.

Errore comune: namespace e cartella incoerenti

Se dichiari:

```
<?php  
namespace App\Model;
```

ma metti il file in una cartella non collegata ad `App\`, Composer potrebbe non trovarlo. Quando una classe non viene caricata, controlla nome del namespace, nome della classe, percorso del file e configurazione `psr-4`.

Composer

Come installare librerie PHP e gestire dipendenze di progetto.

Cosa è Composer

Composer è il gestore di dipendenze più usato in PHP. Una dipendenza è una libreria esterna che il tuo progetto usa per non riscrivere tutto da zero.

Esempio: invece di scrivere da solo una libreria per inviare email, puoi installarne una già pronta.

composer.json

Il file `composer.json` descrive il progetto e le librerie richieste.

Puoi crearne uno con:

```
composer init
```

Per iniziare puoi anche installare direttamente un pacchetto.

```
composer require monolog/monolog
```

Composer aggiorna `composer.json` e scarica il codice nella cartella `vendor`.

Installare dipendenze

Se scarichi un progetto che ha già `composer.json`, esegui:

```
composer install
```

Composer legge anche `composer.lock`, se presente, e installa le versioni previste.

Autoload

Composer crea un file di autoload che carica classi e librerie.

```
<?php  
require __DIR__ . '/vendor/autoload.php';
```

Dopo questa riga puoi usare le librerie installate.

Aggiornare con cautela

Per aggiornare le librerie:

```
composer update
```

Questo comando può cambiare versioni. Nei progetti reali va usato con attenzione e seguito da test.

Regola pratica

- `composer require` aggiunge una libreria
- `composer install` installa cio che il progetto richiede
- `composer update` aggiorna le versioni consentite

Composer diventa importante quando il progetto smette di essere un singolo file e inizia a usare librerie esterne.

composer.lock

Quando installi dipendenze, Composer crea spesso anche `composer.lock`. Questo file registra le versioni esatte installate.

Il suo scopo e semplice: se un'altra persona installa il progetto, riceve le stesse versioni delle librerie. Così si evitano differenze difficili da capire.

Cartella vendor

La cartella `vendor` contiene il codice scaricato da Composer. Di solito non la modifichi a mano.

In molti progetti `vendor` non viene salvata in Git. Si salva invece `composer.json` e `composer.lock`, poi ogni persona esegue:

```
composer install
```

Usare una libreria installata

Dopo l'autoload, puoi usare classi fornite dai pacchetti.

```
<?php
require __DIR__ . '/vendor/autoload.php';

// Qui puoi usare le librerie installate con Composer
```

Ogni libreria ha la propria documentazione, ma il punto di ingresso nel progetto PHP resta quasi sempre `vendor/autoload.php`.

Regola pratica

Non installare librerie per ogni piccola cosa. Prima capisci il problema. Poi scegli una libreria quando ti evita lavoro complesso o delicato.

Test e buone pratiche

Come scrivere PHP piu affidabile, leggibile e facile da mantenere.

Scrivere codice leggibile

Il primo passo per avere codice affidabile e renderlo leggibile.

Scegli nomi chiari:

```
<?php
$prezzoFinale = 18.50;
```

E meglio di:

```
<?php
$pf = 18.50;
```

Funzioni piccole

Una funzione piccola e piu facile da capire e provare.

```
<?php
function calcolaSconto(float $prezzo): float {
    if ($prezzo > 50) {
        return 10;
    }

    return 0;
}
```

Questa funzione fa una cosa: decide lo sconto.

Validare gli input

I dati che arrivano da form, URL o file esterni vanno controllati.

```
<?php
$email = $_POST["email"] ?? "";

if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "Email non valida";
    exit;
}
```

La validazione evita errori e migliora la sicurezza.

Un test automatico semplice

Un test automatico verifica che una parte del codice funzioni come previsto.


```
<?php
function somma(int $a, int $b): int {
    return $a + $b;
}

if (somma(2, 3) !== 5) {
    echo "Test fallito";
    exit(1);
}

echo "Test riuscito";
```

Nei progetti veri si usano strumenti come PHPUnit, ma l'idea è la stessa: controllare il risultato atteso.

Sicurezza essenziale

Abitudini importanti:

- valida i dati in ingresso
- usa `htmlspecialchars` quando stampi dati in HTML
- usa query preparate per il database
- non salvare password in chiaro
- non mostrare errori tecnici in produzione

La sicurezza non è un capitolo separato da ricordare alla fine. È una serie di piccole scelte in ogni pagina.

Rendere un test più leggibile

Anche senza strumenti esterni puoi separare preparazione, azione e controllo.

```
<?php
function calcolaTotale(float $prezzo, int $quantita): float {
    return $prezzo * $quantita;
}

$risultato = calcolaTotale(5, 3);

if ($risultato !== 15.0) {
    echo "Test fallito: totale non corretto";
    exit(1);
}

echo "Test riuscito";
```

Il test dice chiaramente cosa si aspettava.

Buone pratiche quotidiane

Non servono regole complicate per migliorare subito il codice. Parti da queste:

- una variabile deve avere un nome comprensibile
- una funzione deve avere un compito principale
- i dati esterni vanno controllati prima di usarli
- gli errori vanno letti, non ignorati
- il codice duplicato spesso merita una funzione

Prima di considerare finita una pagina

Prova almeno un caso normale, un caso vuoto e un caso sbagliato. Per esempio, se ricevi un'email, prova un'email valida, un campo vuoto e un testo che non è un'email.